

Digital Microelectronics - Assignment 1

Simone Rea

1. Step 1

(Point 1)

`sweep_idvd()` implementation was quite straightforward: all I had to do was copying `sweep_idvg()` and swap V_d and V_g

```
# [과제] sweep_idvd(self, vg, voltages)를 직접 추가해 보자.
def sweep_idvd(self, vg, voltages):
    # I run self.current going through all voltages passed and fixing vg
    ids = [self.current(vg, vd) for vd in voltages]
    return pd.DataFrame({"Vd_V": voltages, "Id_A_per_um": ids})
```

After that I just changed the function called in `compare.py` from `sweep_idvg()` to `sweep_idvd()` and also some units in the labels.

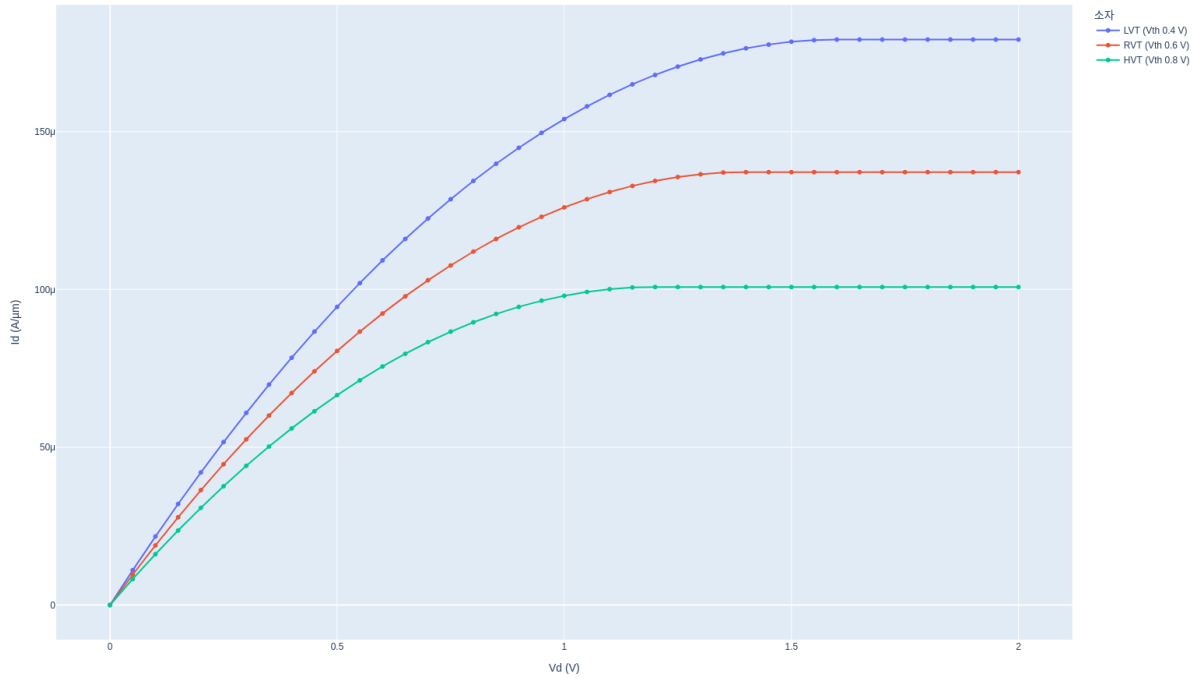
```
curves = []
for name, mosfet in devices.items():
    # 인스턴스 3개를 같은 루프로 돌린다
    # Changed to sweep_idvd
    curve = mosfet.sweep_idvd(vg=2.0, voltages=voltages)

    curve["device"] = name
    curves.append(curve)

result = pd.concat(curves)
result.to_csv("compare_step1.csv", index=False)
print(result.groupby("device")["Id_A_per_um"].max())
print("saved: compare_step1.csv")

# Changed units
fig = px.line(result, x="Vd_V", y="Id_A_per_um", color="device", markers=True,
               title="LVT / RVT / HVT 비교 (Vg = 2 V)",
               labels={"Vd_V": "Vd (V)", "Id_A_per_um": "Id (A/μm)", "device": "소자"})
fig.show()
```

The result is the following:



We can consider the general formula and fix $V_G = 2.0V$ and $k = 1.4 \times 10^{-4}$ since they are the same for all curves. Each curve has different $V_{th} = (0.4V, 0.6V, 0.8V)$ and we sweep through $V_D \in [0.0V, 2.0V]$ with a step of $0.05V$. We can then consider this formula [for formatting reasons I define here $V_{ov} = V_G - V_{th}$]:

$$I_D(V_G, V_D) = \begin{cases} 0 & \text{if } V_{ov} \leq 0 \\ k(V_{ov} - \frac{V_D}{2})V_D & \text{if } 0 < V_D < V_{ov} \\ \frac{1}{2}kV_{ov}^2 & \text{if } V_D \geq V_{ov} \end{cases} \rightarrow \begin{cases} 0 & \text{if } (2.0 - V_{th}) \leq 0 \\ k((2.0 - V_{th}) - \frac{V_D}{2})V_D & \text{if } 0 < V_D < (2.0 - V_{th}) \\ \frac{1}{2}k(2.0 - V_{th})^2 & \text{if } V_D \geq (2.0 - V_{th}) \end{cases}$$

We can see that the point after which the mosfet is saturated increases as V_{th} decreases, and this is obvious considering that the saturation voltage is defined as $V_{ov} = V_G - V_{th}$ (so here $V_{ov} = 2.0V - V_{th}$). Moreover, current depends on V_{ov} and so does its rate of change: we can indeed see that with lower values of V_{th} we both reach higher currents for the same input V_D and current also grows faster as well.

Just considering the formula we get the following results, which obviously respect what the graph shows.

V_{th} (V)	V_{od} (V)	I_{max} (A/μm)
0.4 ■	1.6	1.792×10^{-4}
0.6 ■	1.4	1.372×10^{-4}
0.8 ■	1.2	1.008×10^{-4}

2. Step 2

(Point 2 and 3)

Implementation of `sweep_idvd()` was once again straightforward: just copied `sweep_idvg()` and changed some variables. `sweep_cv()` was a little harder, since you had to use different functions and calculate the gradient, but still it was pretty much identical to other sweep functions.

```
def sweep_idvd(self, vg: float, start: float, stop: float, step: float) -> pd.DataFrame:
    """[과제] 게이트 전압을 고정하고 Vd를 스위칭하여 (Vd, Id) 표를 돌려준다.

    sweep_idvg와 거의 같다 - 어느 접점이 고정되고 어느 접점이 변하는가?
    """

    # We first need to set Gate Voltage to Vg, we go through Vd values
    # from start to stop and for each step we set Drain Voltage to Vd and
    # get Current value Id and finally return

    self.set_bias("gate", vg)
    vds = voltage_points(start, stop, step)
    ids = []
    for vd in vds:
        self.set_bias("drain", float(vd))
        ids.append(self.drain_current())
    return pd.DataFrame({"Vd_V": vds, "Id_A_per_um": ids})
```

```
def sweep_cv(self, start: float, stop: float, step: float) -> pd.DataFrame:
    """[과제] Vg를 스위칭하며 게이트 전하의 기울기 dQg/dVg로 (Vg, Cgg) 표를 돌려준다.

    quasi-static C-V는 enable_transport() 없이 평형 상태에서 계산한다.
    힌트: 전압마다 gate_charge()를 모은 뒤 np.gradient(전하, 전압)로
    기울기를 구하고, * UM 으로 F/um 단위로 바꾼다.
    """

    # We go through Vg values from start to stop and for each step
    # we set Gate Voltage to Vg and get Charge value Qg
    # After that we calculate gradient to find Capacitance Cgg,
    # fix measurement unit multiplying by UM and finally return

    vgs = voltage_points(start, stop, step)
    charges = []
    for vg in vgs:
        self.set_bias("gate", float(vg))
        charges.append(self.gate_charge())

    capacitances = np.gradient(charges, vgs) * UM
    return pd.DataFrame({"Vg_V": vgs, "Cgg_F_per_um": capacitances})
```

Finally I implemented the `run_idvd()` and `run_cv()` functions. Both are very similar to the already implemented `run_idvg()`, only notable difference is that `run_cv()` does not need `sim.enable_transport()` since you don't need to simulate electrons flowing to calculate the charge of the capacitor.

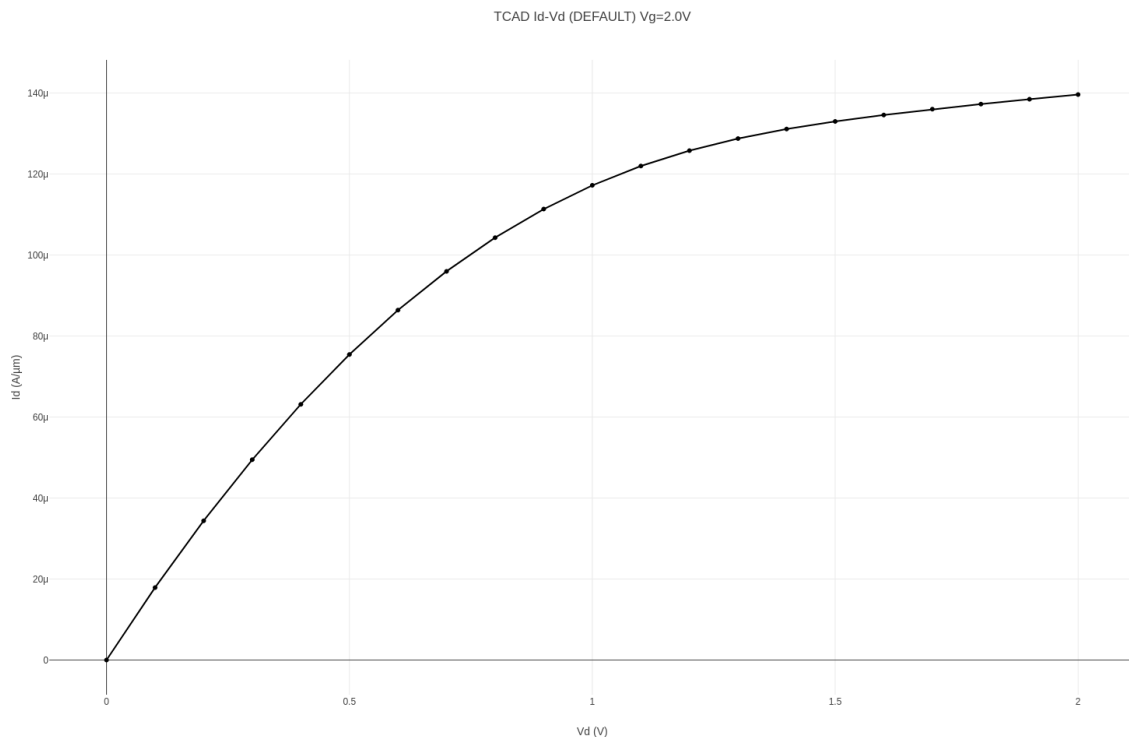
```
def run_idvd(device: Device, gate_v: float = 2.0, start_v: float = 0.0,
            stop_v: float = 2.0, step_v: float = 0.1) -> pd.DataFrame:
    """[과제] run_idvg를 참고해 Id-Vd 해석을 완성하라."""

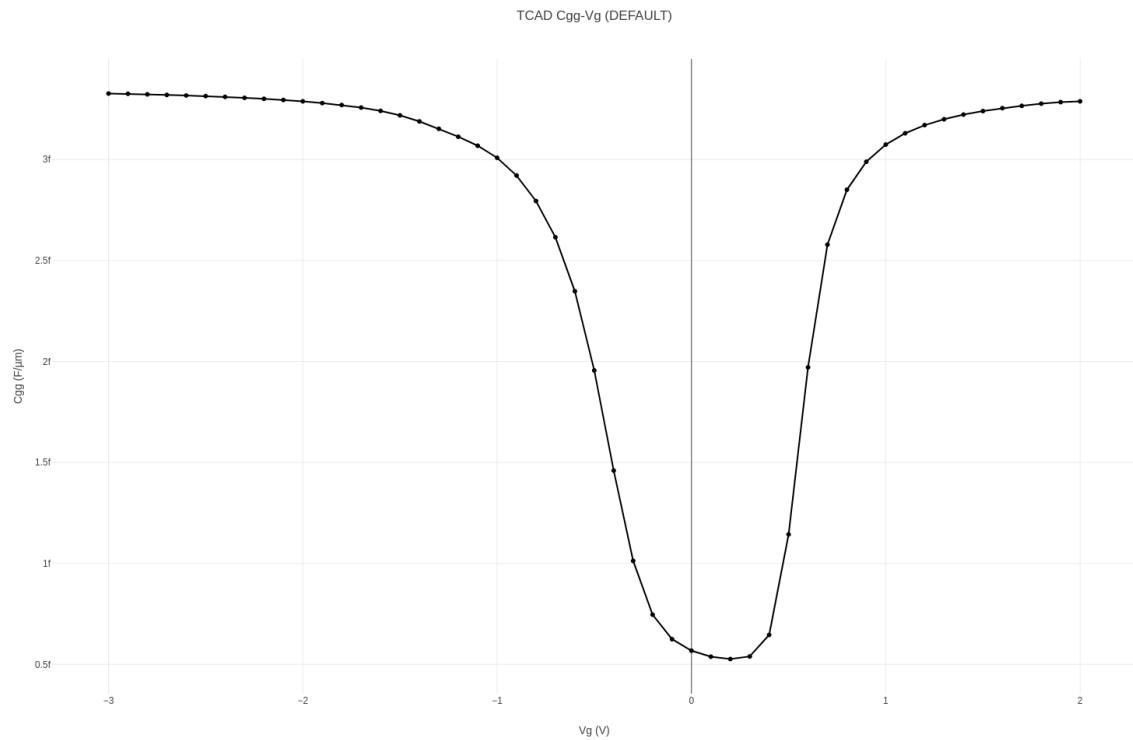
    # Creat and build Simulator, solve equilibrium, enable transport and
    # sweep through vd from starv_v to stop_v with step step_v to calculcate Id
    sim = MosfetSimulator(device, name="idvd")
    sim.build()
    sim.solve_equilibrium()
    sim.enable_transport()
    return sim.sweep_idvd(gate_v, start_v, stop_v, step_v)

def run_cv(device: Device, start_v: float = -1.0, stop_v: float = 2.0,
            step_v: float = 0.1) -> pd.DataFrame:
    """[과제] C-V는 전류가 필요 없다 enable_transport()를 부르지 않는 이유를 생각해 보라."""

    # Creat and build Simulator, solve equilibrium, DO NOT enable transport and
    # sweep through vg from starv_v to stop_v with step step_v to calculate Cgg
    sim = MosfetSimulator(device, name="cv")
    sim.build()
    sim.solve_equilibrium()
    return sim.sweep_cv(start_v, stop_v, step_v)
```

Here is an example of I_d - V_d and C_{gg} - V_g plotted using default values for the device (I executed `run_example.py`):





(Point 4)

For Step 2 we also needed to change `config.yaml` and `compare_tcad.py` to call the right functions and define the values to sweep through.

```
sweeps:
  idvg:
    drain_v: 1.0
    start_v: 0.0
    stop_v: 2.0
    step_v: 0.1
# idvd / cv 스윙 설정은 2단계 과제를 구현한 뒤 여기에 추가해 보자
# Added idvd and cv
  idvd:
    gate_v: 1.0
    start_v: 0.0
    stop_v: 2.0
    step_v: 0.1
  cv:
    start_v: 0.0
    stop_v: 2.0
    step_v: 0.1
```

```

curves = []
for name, overrides in variants.items():
    # 기본 소자에서 compare:에 적힌 옵션만 덮어쓴 새 소자를 만든다
    device = dataclasses.replace(base_device, **overrides)
    print(f"\n==== {name} 시뮬레이션 =====", flush=True)

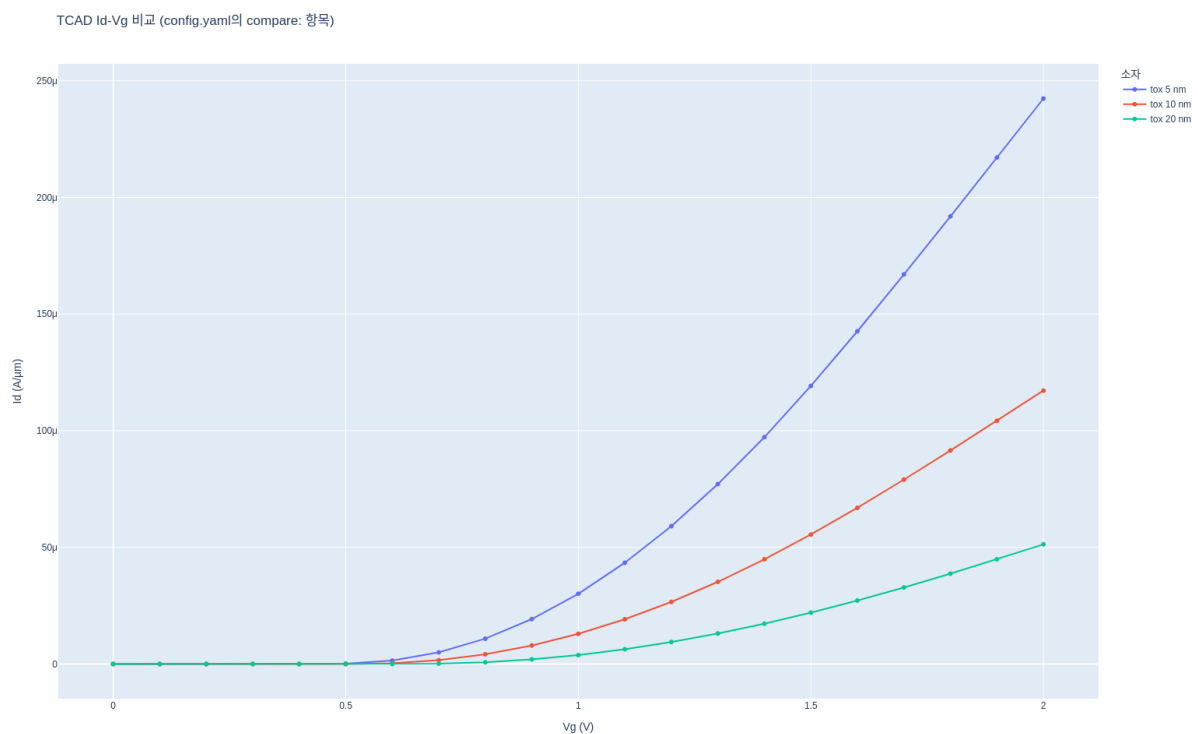
    # I execute run_idvg, runvd and run_cv and add curves
    # to a single array
    curve_idvg = run_idvg(device, **sweeps.get("idvg", {}))
    curve_idvd = run_idvd(device, **sweeps.get("idvd", {}))
    curve_cv = run_cv(device, **sweeps.get("cv", {}))

    curve_idvg["device"] = name
    curve_idvd["device"] = name
    curve_cv["device"] = name

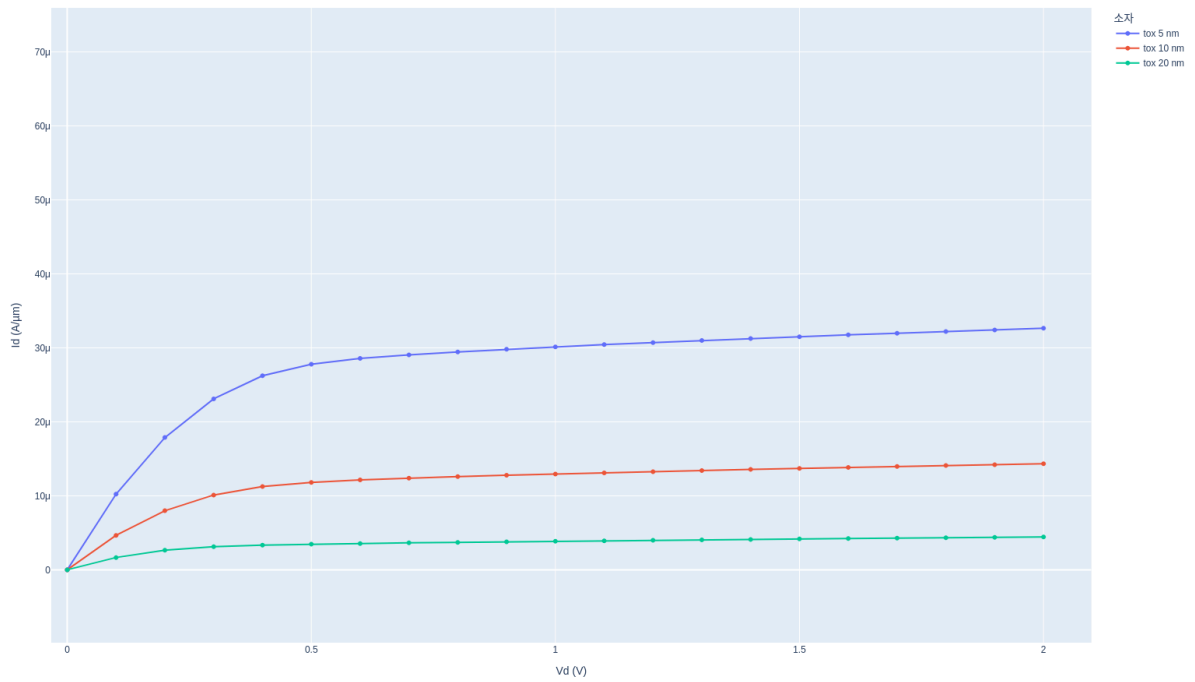
    curves.append(curve_idvg)
    curves.append(curve_idvd)
    curves.append(curve_cv)

```

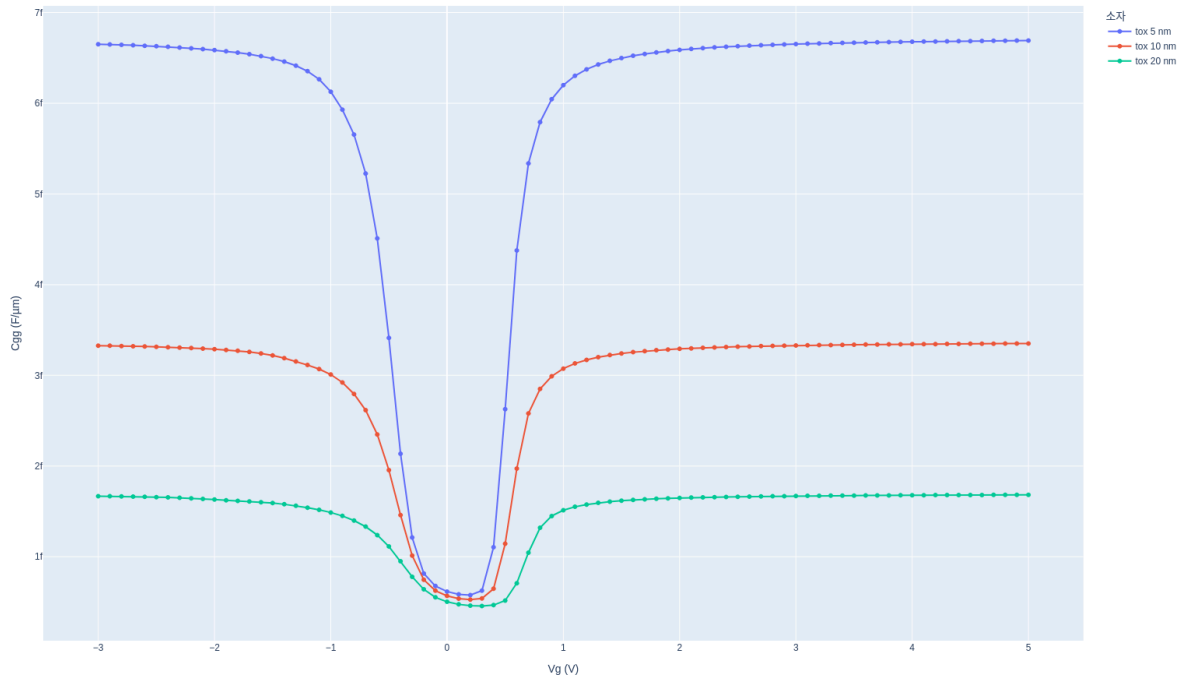
Plotting the results from the run functions we get:



TCAD Id-Vd 비교 (config.yaml의 compare: 항목)



TCAD Cgg-Vg 비교 (config.yaml의 compare: 항목)



Considerations:

- I_d - V_g : The curves start to grow after some certain amount of voltage V_g applied to gate, which is the threshold voltage V_{th} . This depends on the physical structure of the Mosfet, so it makes sense that changing the thickness of the oxide layer changes the value of V_{th} : we can see this by looking at the exact V_g at which the curve starts to grow which shifts between different curves. Thinner oxide layers also lead to higher currents once the device is on: since $C_{ox} = \epsilon_{ox} \frac{A}{t_{ox}}$ increases as

t_{ox} shrinks, the gate induces more inversion charge for the same overdrive voltage $V_g - V_{\text{th}}$, and the transconductance parameter $k \propto C_{\text{ox}}$ increases accordingly, giving a steeper, higher current curve. In conclusion, k gets bigger as the oxide thickness decreases, which is why the thinnest device (tox 5nm) shows both the earliest turn-on and the steepest, highest current among the three curves. The general formula used for current is still

$$I_D(V_G, V_D) = \begin{cases} 0 & \text{if } V_{\text{ov}} \leq 0 \\ k(V_{\text{ov}} - \frac{V_D}{2})V_D & \text{if } 0 < V_D < V_{\text{ov}} \\ \frac{1}{2}kV_{\text{ov}}^2 & \text{if } V_D \geq V_{\text{ov}} \end{cases}$$

with $V_D = 1.0V$. Interesting to notice that we pass through Saturation at beginning when $V_D \geq V_G - V_{\text{th}}$, even though we don't really see it since the current always remains a function of V_G .

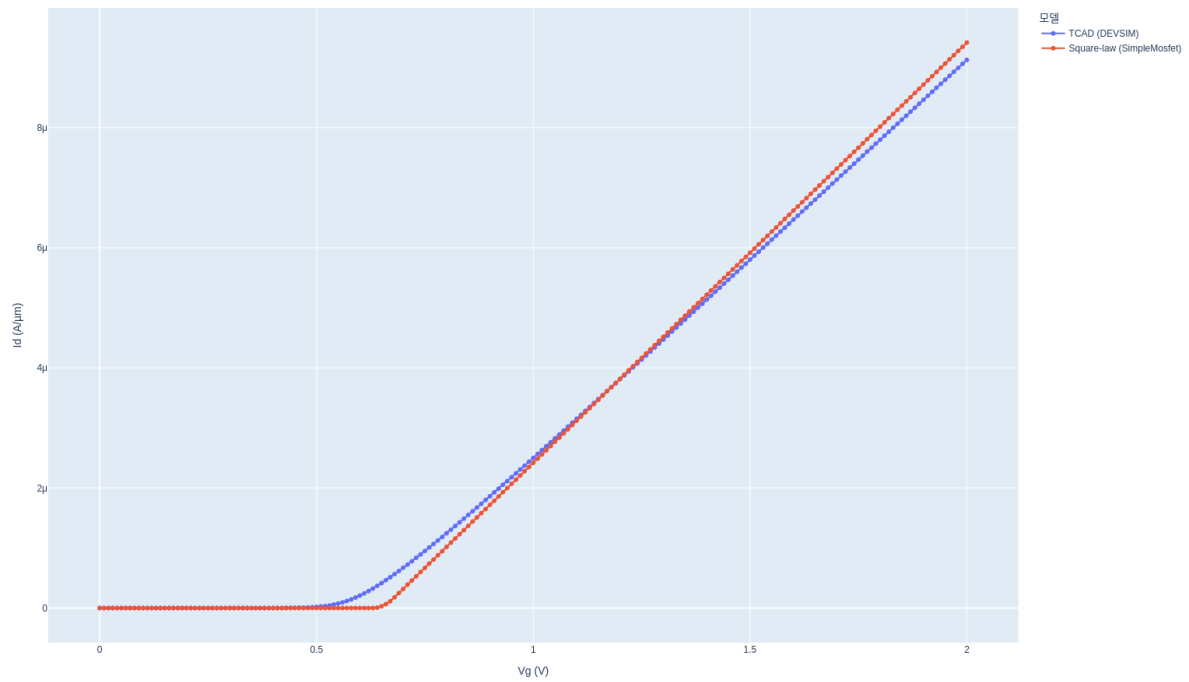
- I_d - V_d : Most of what we said in Step 1 still applies here, but there is one key difference: in the square-law model, current is exactly flat once the device enters Saturation, whereas the TCAD curve keeps rising slightly even past pinch-off. This is because the simple model treats the channel length as fixed, while DEVSIM captures Channel Length Modulation: as V_d increases beyond V_{dsat} , the pinch-off point moves further toward the source, shortening the effective channel length and allowing a bit more current to flow even though the device is nominally in Saturation.
- C_{gg} - V_g : Here we see the capacitance of the Gate. Before the channel is created ($V_g < V_{\text{th}}$), the gate voltage only widens the depletion region beneath the oxide. This depletion region behaves like a second capacitor, C_{dep} , in series with C_{ox} , and the two combine as $C_{\text{gg}} = \frac{C_{\text{ox}}C_{\text{dep}}}{C_{\text{ox}} + C_{\text{dep}}}$. Since a series combination is always smaller than either individual capacitance, $C_{\text{gg}} < C_{\text{ox}}$ throughout this region, which is why the curve stays low and roughly flat while V_g is below V_{th} . Once V_g crosses V_{th} , the channel forms and the capacitance rises sharply, finally stabilizing near C_{ox} in strong inversion, since the inversion layer now screens the depletion region entirely and the structure behaves like an ideal parallel-plate capacitor again. A thinner oxide layer corresponds to a bigger capacitance, since $C_{\text{ox}} = \epsilon_{\text{ox}} \frac{A}{t_{\text{ox}}}$.

(Point 5)

Finally we compare the TCAD simulator with the naive implementation from Step 1 looking at the differences between the `sweep_idvg()` functions. We set $V_d = 0.05V$, $V_{\text{th}} = 0.6295$ and $k = 1.4 \times 10^{-41}$

1. The value of k and V_{th} were calculated giving device's parameters in `config.yaml` to Claude

Square-law vs TCAD (Vd = 0.05 V, Vth_matched = 0.6295 V)



The biggest notable difference can be seen near V_{th} : since the Square-Law model uses a simple formula that evaluates to zero when $V_g < V_{th}$, we get an instant rising of I_d when we surpass it. On the other hand, the DEVSIM model simulates charges moving and so current rises gradually and more smoothly. Moreover, working with fixed values of k and V_{th} instead of values derived directly from the physical dimensions gets us some discrepancy between the two curves.

To compare the two models I used the following code:

```

from __future__ import annotations

from pathlib import Path

import numpy as np
import pandas as pd
import plotly.express as px

from step1.simulator_practice import SimpleMosfet
from step2.mosfet_tool.config import load_config
from step2.mosfet_tool.workflows import run_idvg

# 1단계: square-law 모델
# 2단계: 실제 소자 설정
# 2단계: DEVSIM TCAD 워크플로

ROOT = Path(__file__).resolve().parent

# -----
# 1) TCAD 실제 물리 모델 □ config.yaml의 기본 소자(base device) 사용
# -----
base_device, sweeps = load_config(ROOT / "step2/config.yaml")
idvg_settings = sweeps.get("idvg", {}) # drain_v, start_v, stop_v, step_v

start_v = 0.0
stop_v = 2.0
drain_v = 0.05
step_v = 0.1

print("===== TCAD (DEVSIM) 시뮬레이션 =====", flush=True)
curve_tcad = run_idvg(base_device, start_v=start_v, stop_v=stop_v, drain_v=drain_v, step_v=step_v)
curve_tcad["model"] = "TCAD (DEVSIM)"

# -----
# 2) Square-law 모델 □ "같은 소자"를 근사하기 위해 TCAD 곡선에서
# 추출한 Vth를 사용한다 (extract_vth.py 참고: tox=10nm 소자는 Vth □ 0.63 V)
# -----
VTH_MATCHED = 0.6295 # TODO: 다른 base_device를 쓴다면 이 값을 다시 추출해서 갱신할 것
K_MATCHED = 1.4e-4

voltages = np.arange(start_v, stop_v + 1e-9, step_v)

print("===== Square-law (SimpleMosfet) =====", flush=True)
simple_mosfet = SimpleMosfet(vth=VTH_MATCHED, k=K_MATCHED)
curve_simple = simple_mosfet.sweep_idvg(vd=drain_v, voltages=voltages)
curve_simple["model"] = "Square-law (SimpleMosfet)"

# -----
# 3) 합쳐서 하나의 그래프에 겹쳐 그리기
# -----
result = pd.concat([
    curve_tcad[["Vg_V", "Id_A_per_um", "model"]],
    curve_simple[["Vg_V", "Id_A_per_um", "model"]],
])
result.to_csv("compare_models.csv", index=False)
print("saved: compare_models.csv")

fig = px.line(result, x="Vg_V", y="Id_A_per_um", color="model", markers=True,
    title=f"Square-law vs TCAD (Vd = {drain_v} V, Vth_matched = {VTH_MATCHED} V)",
    labels={"Vg_V": "Vg (V)", "Id_A_per_um": "Id (A/μm)", "model": "모델"})
fig.show()

```